🎬 Video Presentation Script & Guide

Experiment 3: Sampling, Aliasing & Whittaker-Shannon Sinc Reconstruction

Presenter: Apurba Maity (Roll No: `34900324001`)

Department: Electronics & Communication Engineering $|$ Cooch Behar GEC

⏱️ **Duration: 4:30 – 5:00 min** 🎯 **Focus: Python Code & Plot Walkthrough**

🌟 **Theory vs Practicality Deep-Dive**

📋 Pre-Recording Windows & Setup

🖥️	**Window 1 (Main Focus — 75% of video):**	VS Code / Jupyter Notebook with `experiment_03.py` and `Experiment_03_Lab_Report.ipynb`.
📊	**Window 2 (Plots Viewer):**	Matplotlib generated figures in `plots/` directory.
🌐	**Window 3 (Closure — 30 seconds):**	Interactive Web Studio (`index.html`) in full-screen browser.
🐙	**Window 4 (Closing):**	GitHub Public Repository (`SecretiveCodeRunner/Digital_Communication_Lab`).

🟢 SCENE 1: Introduction & Objectives (0:00 – 0:40)

> 🖥️ **ON-SCREEN ACTION**
>
> Show VS Code open to `Experiment_03_Lab_Report.ipynb` on the title banner, displaying your Name, Roll Number, and Subject Header.

> 🎙️ **SPOKEN NARRATION (SPEAK CLEARLY AND STEADILY)**
>
> "Hello everyone and respected faculty. My name is **Apurba Maity**, Roll Number **34900324001**, from the Department of Electronics and Communication Engineering at Cooch Behar Government Engineering College.
>
> Today, I will be presenting **Experiment 3** of our Software-Based Digital Communication Laboratory: **Sampling, Aliasing, and Whittaker-Shannon Sinc Reconstruction**.
>
> Our core objective today is to explore the **Nyquist-Shannon Sampling Theorem** through Python numerical simulation, implement **ideal sinc interpolation** from first principles, observe **spectral aliasing** when undersampled, and most importantly, examine the critical differences between **pure theoretical mathematical assumptions and practical computational realities**."

🔵 SCENE 2: Theoretical Formulation & Signal Model (0:40 – 1:25)

> 🖥️ **ON-SCREEN ACTION**
>
> Scroll to Section 1 of the Notebook, highlighting the mathematical formulas for $x(t)$, f_{Nyquist}, and the aliasing folding formula.

> 🎙️ **SPOKEN NARRATION**
>
> "Let us first establish our theoretical framework. We construct a deterministic continuous two-tone signal:
>
> $$x(t) = \sin(2\pi \cdot 100t) + 0.5\sin(2\pi \cdot 300t)$$
>
> Here, the fundamental frequency is $f_1 = 100\text{ Hz}$ and the highest harmonic is $f_2 = 300\text{ Hz}$. Thus, the maximum signal bandwidth is $f_{\max} = 300\text{ Hz}$.
>
> According to the **Nyquist-Shannon Sampling Theorem**, exact lossless recovery requires a sampling frequency satisfying:
>
> $$f_s \geq 2f_{\max} = 2 \times 300 \text{ Hz} = 600 \text{ Hz}$$

Continuous-time reconstruction is governed by the **Whittaker-Shannon sinc interpolation formula**:

$$\hat{x}(t) = \sum_{n=-\infty}^{\infty} x[n] \cdot \text{sinc}\left(f_s(t - nT_s)\right), \quad \text{where } \text{sinc}(u) = \frac{\sin(\pi u)}{\pi u}$$

If we violate this by undersampling at $f_s = 400\text{ Hz}$, the high-frequency 300 Hz component folds across the Nyquist boundary $f_s/2 = 200\text{ Hz}$ according to the aliasing formula:

$$f_{\text{alias}} = |f_2 - k \cdot f_s| = |300 - 1 \times 400| = |-100| = 100 \text{ Hz}$$

Now, let us examine how we implemented this in Python without using external black-box toolboxes."

🟡 SCENE 3: Python Code Architecture & DSP Walkthrough (1:25 – 2:15)

🖥️ ON-SCREEN ACTION

Switch to `experiment_03.py` in VS Code. Highlight the `sinc_reconstruction()` function (lines 53–63) and `calculate_spectrum()` (lines 65–73).

```
def sinc_reconstruction(sample_times, sample_values, t_fine, fs):
    # Vectorized 2D broadcast: shape (N_samples, M_eval_points)
    dt_matrix = t_fine[None, :] - sample_times[:, None]
    sinc_matrix = np.sinc(fs * dt_matrix)  # np.sinc computes sin(pi*x)/(pi*x)
    # Sum overlapping sinc pulses across all discrete samples
    x_hat = np.dot(sample_values, sinc_matrix)
    return x_hat
```

🎙️ SPOKEN NARRATION

"Here in our Python implementation, we avoided black-box DSP functions and implemented the core mathematics directly using NumPy.

As shown in lines 53 to 63 in `sinc_reconstruction`:

- To evaluate the Whittaker-Shannon summation efficiently for thousands of time points, we construct a 2D distance matrix using array broadcasting: `dt_matrix = t_fine[None, :] - sample_times[:, None]`.
- Passing this into `np.sinc(fs * dt_matrix)` creates our full interpolation kernel grid.
- A single vectorized dot product `np.dot(sample_values, sinc_matrix)` instantly sums all overlapping sinc pulses across the entire continuous evaluation grid in a fraction of a millisecond.

For frequency analysis, lines 65 to 73 use `np.fft.rfft` to compute the single-sided discrete Fourier spectrum, normalized by $2/N$ to obtain exact physical peak amplitudes.

Furthermore, in lines 110 to 125, we implemented sample boundary padding ($N_{\text{pad}} = 20$), eliminating artificial edge-truncation errors in our numerical reconstruction."

🟣 SCENE 4: Plot Walkthrough & Empirical Validation (2:15 – 3:25)

🖥️ ON-SCREEN ACTION

Display the 4 generated Matplotlib figures from `plots/` or scroll through Section 3 of the Jupyter Notebook.

🎙️ SPOKEN NARRATION

"Let us now examine our empirical results across the 4 generated figures:

1. Discrete Sample Stems (Figure 1): Over our 50 ms test duration:

- At **1800 Hz** (top), 91 dense samples capture 6 samples per cycle of the 300 Hz wave.
- At **600 Hz** (middle), 31 samples capture exactly 2 samples per cycle.
- At **400 Hz** (bottom), only 21 samples are recorded—barely 1.3 samples per cycle—missing the sinusoidal peaks.

2. Reconstructed Waveforms & FFT Spectra (Figures 2 & 3):

- In the **1800 Hz Oversampled case**, the reconstructed dashed curve overlays the original black reference signal perfectly, yielding a near-zero Mean Squared Error of $5.0 \times 10^{-6}\text{ V}^2$. The FFT spectrum cleanly recovers both the 100 Hz and 300 Hz peaks.
- In the **400 Hz Undersampled case**, the reconstructed waveform is heavily distorted. Looking at the FFT spectrum, the 300 Hz peak has completely vanished from its true position and folded directly onto **100 Hz**, exactly confirming our theoretical derivation $|300 - 400| = 100\text{ Hz}$!

3. Time-Domain Error (Figure 4): The error waveform $e(t) = x(t) - \hat{x}(t)$ confirms this quantitatively: oversampling yields a flat zero line, while undersampling results in a massive oscillating error with $\text{MSE} = \mathbf{0.25\text{ V}^2}$."

🟠 SCENE 5: 🌟 Theory vs. Practicality & Computational Realities (3:25 – 4:15)

🖥️ ON-SCREEN ACTION

Scroll to **Section 4: Theory vs. Practicality** in the Notebook. Point with mouse cursor to the comparative table and error numbers.

💡 KEY DEFENSE POINT

Explain why $f_s = 600\text{ Hz}$ produced $\text{MSE} = 0.1235$ despite being at the Nyquist rate, and discuss finite-window sinc truncation and hardware anti-aliasing filters.

🎙️ SPOKEN NARRATION

"Now, let us address the most insightful question of this experiment: **Where does pure mathematical theory diverge from practical simulation and physical hardware?**

1. Phase Nulling at Critical Nyquist ($f_s = 2f_{\max} = 600\text{ Hz}$):
In textbooks, theory states that $f_s = 2f_{\max}$ guarantees exact signal reconstruction. However, in our simulation, the 600 Hz critical case produced an MSE of **0.1235**! Why did this happen?

Our test signal is a pure sine wave: $\sin(2\pi \cdot 300 t)$. When sampled at intervals of $T_s = 1/600\text{ s}$, every sample falls exactly on a zero crossing:

$$x_{\text{harmonic}}[n] = 0.5 \sin\left(2\pi \cdot 300 \cdot \frac{n}{600}\right) = 0.5 \sin(n\pi) \equiv 0$$

Every single sample of the 300 Hz tone evaluated to zero! The sinc interpolator was therefore blind to the 300 Hz wave and only reconstructed the 100 Hz tone. The missing harmonic power is exactly $\frac{A_2^2}{2} = \frac{0.5^2}{2} = \mathbf{0.125\text{ V}^2}$, which matches our measured MSE of 0.1235 V^2!

This proves that in real-world DSP and ADC design, we cannot operate right at the theoretical boundary; we must strictly oversample ($f_s \ge 2.2 f_{\max}$) with a safety guard band to avoid phase nulling.

2. Infinite Sinc Summation vs. Finite Observation Window:
Theoretical Whittaker-Shannon interpolation requires an infinite summation from $-\infty$ to $+\infty$. Real DSP processors and digital scopes operate on finite time windows ($T = 50\text{ ms}$). Truncating the sinc series introduces boundary ripples, which we resolved by implementing sample padding ($N_{\text{pad}} = 20$).

3. Physical Anti-Aliasing Filtering (AAF):
In software math, we can inspect and predict folded frequencies. But in physical ADC hardware, once an out-of-band high-frequency signal aliases into baseband, it is mathematically inseparable from legitimate data. Therefore, an analog active Low-Pass Anti-Aliasing Filter must always precede the hardware ADC."

🔘 SCENE 6: Interactive Web Simulator (Bonus Closure) (4:15 – 4:45)

🖥️ ON-SCREEN ACTION

Switch to browser window displaying `index.html`. Click preset buttons (**1800 Hz**, **600 Hz**, **400 Hz**) and briefly drag the f_s slider.

🎙️ SPOKEN NARRATION

"As a bonus interactive closure, we also compiled this lightweight 60 FPS HTML5 simulation studio.

Here, we can dynamically drag the sampling frequency slider from 1800 Hz down to 300 Hz. Notice how the reconstructed green waveform morphs in real time, and in the live FFT spectrum below, you can watch the aliased red peak sweep continuously across the folding boundary as f_s decreases. We can also synthesize Web Audio tones to hear how aliasing lowers the perceived auditory pitch."

🔴 SCENE 7: Summary & Conclusion (4:45 – 5:00)

🖥️ ON-SCREEN ACTION

Switch to GitHub repository page (`SecretiveCodeRunner/Digital_Communication_Lab`).

🎙️ SPOKEN NARRATION

"In conclusion, this experiment successfully verified the Nyquist-Shannon Sampling Theorem, implemented vectorized sinc reconstruction in Python, validated spectral aliasing folding, and clarified the boundary between continuous theory and discrete DSP implementation.

The complete Python code, Jupyter lab notebook, printable study guides, and the interactive studio are published on my open-source GitHub repository.

Thank you very much for your time and attention!"

📊 Summary Data Table & Faculty Viva Defense

Regime	f_s (Hz)	Ratio	Samples	MSE (V^2)	RMSE (V)	Harmonic Status
Oversampled	1800	$3.0\times$	91	5.0×10^{-6}	0.0022	Clean recovery at 300 Hz
Critical	600	$1.0\times$	31	0.1235	0.3514	Phase Null ($\sin(n\pi)=0$)
Undersampled	400	$0.67\times$	21	0.2500	0.5000	**Aliased to 100 Hz**

🎯 Predictive Viva Q&A Defense

Q1: Why did Critical Sampling ($f_s = 600\text{ Hz}$) yield an MSE of 0.1235 instead of 0?
Answer: The 300 Hz harmonic was $0.5\sin(2\pi \cdot 300 t)$. At $t_n = n/600$, $\sin(n\pi) \equiv 0$. All samples were zero, so sinc interpolation missed the 300 Hz tone. The energy of the lost harmonic is $\frac{1}{2} A_2^2 = \frac{1}{2}(0.5)^2 = 0.125\text{ V}^2$, matching our measured MSE of 0.1235 V^2.

Q2: How does Python evaluate sinc reconstruction so fast?
Answer: Vectorized 2D distance matrix `dt = t_fine[None, :] - sample_times[:, None]` and matrix product `np.dot(sample_values, sinc_matrix)`.

Q3: Why is an Anti-Aliasing Filter essential in hardware ADCs?
Answer: Once an out-of-band signal aliases into baseband, it is mathematically inseparable from true data; analog low-pass filtering must precede sampling.

Digital Communication Laboratory $|$ Apurba Maity (Roll: 34900324001) $|$ Cooch Behar Government Engineering College